SLIDE 1 / 46 · 20 seconds

Behind the flags

WHAT TO SAY

“These archives tell the stories of challenges solved during the CTF. We’ll revisit the reasoning, one case at a time, using diagrams and the evidence that was saved. We’ll start with a very visual example: text turned into numbers, then recovered using GPT-2’s public tables.”

ANIMATION CUES

1. Introduce the challenge.
2. → Reveal the next idea.
3. → Close with the evidence or the lesson.

BACKGROUND

LIKELY QUESTIONS

SOURCES

Challenge journals, the Tapo PowerPoint, and the supplied files.

LIMITATIONS AND PROVENANCE

Teaching diagrams based on the files. Animation timings do not represent the actual time taken to solve the challenges.

Embeddings — the challenge

WHAT TO SAY

“We’re given a file called Dockerfile.json. Despite its name, it contains JSON and numbers. It identifies the model as GPT-2. There are 24 groups of 768 numbers. The flag has been converted into this representation. Our task is to recover the text behind it.”

ANIMATION CUES

1. Introduce the challenge.
2. → Reveal the next idea.
3. → Close with the evidence or the lesson.

BACKGROUND

The file contains session_id, model="gpt2", and inputs_embeds. The 24 × 768 shape can be checked directly in the JSON. A vector is simply a list of numbers. A token is a piece of text: sometimes a word, sometimes a few letters, sometimes a symbol.

LIKELY QUESTIONS

Do you need to know machine learning? No: a lookup table and addition are enough. The diagram explains the structure confirmed by the solution, rather than treating an initial hypothesis as an established fact.

SOURCES

LOST_IN_EMBEDDING · JOURNAL 20:05, 20:09 · DOCKERFILE.JSON

LIMITATIONS AND PROVENANCE

Teaching diagrams based on the files. Animation timings do not represent the actual time taken to solve the challenges.

SLIDE 3 / 46 · 45–60 seconds

Embeddings — the dictionary

WHAT TO SAY

“Here, an embedding is a list of numbers associated with a piece of text. Think of a dictionary: ‘BA’ on the left, 768 numbers on the right. GPT-2 has this table. In our file, a second list of numbers has been added to indicate the token’s position in the text. Both tables are available in the public model.”

ANIMATION CUES

1. Introduce the challenge.
2. → Reveal the next idea.
3. → Close with the evidence or the lesson.

BACKGROUND

The tables are wte, with 50,257 token rows, and wpe, with positions. In this challenge, vecteur[i] = wte[token_i] + wpe[i]. The tokens BA, 26, and { are indeed the first three tokens recovered (4339, 2075, 90). The bars are an illustration, not a computed projection of the 768 dimensions.

LIKELY QUESTIONS

Is a token a letter? Not necessarily. The first three are BA, 26, and {. So don’t call them 24 letters: they are 24 pieces of text. The tables were downloaded during the CTF with permission; nothing was downloaded for this presentation.

SOURCES

LOST_IN_EMBEDDING · CLAUDE/WRITEUP.MD, SECTIONS 2–4

LIMITATIONS AND PROVENANCE

Teaching diagrams based on the files. Animation timings do not represent the actual time taken to solve the challenges.

SLIDE 4 / 46 · 45–60 seconds

Embeddings — reversible addition

WHAT TO SAY

“Let’s use three numbers so the idea fits on one screen. The token is two, five, one. Its position is one, zero, three. Adding column by column gives three, five, four. We know the position, so we subtract it and recover two, five, one. All that remains is to find that row in the dictionary. In the actual file, it’s the same operation on 768 numbers.”

ANIMATION CUES

1. Show the two lists to be added.
2. → The received numbers and the known position.
3. → Recover the original row.
4. → Extend the idea to 768 dimensions.

BACKGROUND

An example constructed to explain vector addition. These values represent neither a real GPT-2 token nor a row from the file. The confirmed structure is E[i] = wte[token_i] + wpe[i], with rounding differences.

LIKELY QUESTIONS

Why not ask a chatbot? The task is to find rows in tables, not generate a plausible sentence.

SOURCES

TEACHING EXAMPLE · STRUCTURE CONFIRMED IN CLAUDE/WRITEUP.MD

LIMITATIONS AND PROVENANCE

The values are fictional; the calculation’s structure matches the challenge.

SLIDE 5 / 46 · 45–60 seconds

Embeddings — the reverse path

WHAT TO SAY

“Take the first group. We know it contains the token’s numbers plus its position’s numbers. Subtract the position numbers, and what remains is almost exactly a row in the dictionary. Find the closest row: it corresponds to ‘BA’. Then do the same for the second group, the third, and so on.”

ANIMATION CUES

1. Read a group of numbers.
2. → Subtract its position component.
3. → Find the closest token.
4. → Repeat and assemble.

BACKGROUND

The agent loaded GPT-2’s public weights, subtracted wpe for positions 0 through 23, then compared each vector with all 50,257 rows of wte using Euclidean distance. The recovered indices were passed to the tokenizer to reconstruct the string.

LIKELY QUESTIONS

Why “closest” rather than “identical”? The saved numbers have very small differences. After correction, the observed distances range from 0.000 to 0.004. Nearest-neighbour lookup is enough. No text is generated, and no prompt is sent to a chatbot.

SOURCES

LOST_IN_EMBEDDING · CLAUDE/SRC/INVERT.PY · CLAUDE/OUT-01.TXT

LIMITATIONS AND PROVENANCE

Teaching diagrams based on the files. Animation timings do not represent the actual time taken to solve the challenges.

SLIDE 6 / 46 · 45–60 seconds

Embeddings — verify rather than guess

WHAT TO SAY

“The agent first compares the numbers directly with the table: the flag already appears. But the distances remain high. After subtracting the position, the same tokens are recovered and the distances become almost zero. This correction explains the numbers we received, as well as producing plausible text.”

ANIMATION CUES

1. Compare the raw vectors.
2. → Subtract the position: the distances drop.
3. → The recovered text stays the same.
4. → Conclude with the almost exact matches.

BACKGROUND

The saved output gives Euclidean distances of 3.5 to 9.9 for the raw vectors and 0.000 to 0.004 after subtracting wpe. Cosine similarities rise from 0.35–0.74 to 1.000. The bars are schematic and do not share a quantitative scale.

LIKELY QUESTIONS

Had the first method failed? No: it already produced the correct text. The correction confirms the structure.

SOURCES

LOST_IN_EMBEDDING · CLAUDE/OUT-01.TXT · CLAUDE/WRITEUP.MD §4

LIMITATIONS AND PROVENANCE

Measurements read from the saved output. No search was rerun.

Embeddings — the result

WHAT TO SAY

“Joining the pieces gives us the flag. The saved output shows it, and the measurements confirm almost exact matches. The lesson: replacing text with its embeddings did not make it secret here. We knew the model, its tables were public, and the operation could be reversed.”

Optional light-hearted aside: “It changed its outfit, but left its name in the cloakroom.”

ANIMATION CUES

1. Introduce the challenge.
2. → Reveal the next idea.
3. → Close with the evidence or the lesson.

BACKGROUND

The raw result, before subtracting positions, already gave the correct tokens, with much larger distances. The correction makes the match convincing: distances of 0.000 to 0.004 and cosine similarities displayed as 1.000. The final script is claude/src/invert.py; the output is claude/out-01.txt.

LIKELY QUESTIONS

Does this apply to all embeddings? This demonstration concerns these GPT-2 input embeddings, formed from a token table and a position table. Do not present this method as a universal way to invert every vector from every model.

SOURCES

LOST_IN_EMBEDDING · JOURNAL 20:09 · CLAUDE/OUT-01.TXT

LIMITATIONS AND PROVENANCE

The high-resolution screenshot was produced after the event from the saved output. The solution was not rerun for this presentation.

SLIDE 8 / 46 · 45–60 seconds

ID, please — two ways in

WHAT TO SAY

“The site offers password login or Google login. With Google login, a statement is passed to the site identifying the account that is signing in. That statement should be verified. But the path called ‘legacy’ even accepts an unsigned statement. Its contents can therefore be fabricated. That still isn’t enough, though: it has to identify the right account.”

ANIMATION CUES

1. Introduce the two login methods.
2. → Explain the Google identity statement in simple terms.
3. → Reveal the missing signature verification.

BACKGROUND

The journals describe POST /member/auth/socialLogin, which accepts an unsigned JWT with alg:none and responds with “legacy compatibility login”. JWT is the statement’s format. The sub field contains the Google account ID. The application then creates a session using an opaque cookie, separate from this JWT. The format’s details are not needed to follow the demonstration.

LIKELY QUESTIONS

Why a signature? It lets the site check the statement’s origin and integrity. Reading an ID from a document that a visitor can fabricate does not prove that the visitor owns the corresponding account.

SOURCES

ID_PLEASE · JOURNAL 18:39, 20:25 · WRITEUP, SECTIONS 1–2

LIMITATIONS AND PROVENANCE

Teaching diagram: no login screenshot is presented as evidence. The analysis uses saved observations; no request was rerun.

ID, please — the wrong identity

WHAT TO SAY

“The first attempts were intuitive: write ‘I am admin’ in the document, or use the admin’s email. They failed. The role came from the account already stored in the database, and the site found that account using a field called sub. Same email but a different sub: a different account, with regular user privileges. So the real question becomes: how do we find the admin account’s ID?”

Optional light-hearted aside: “I appointed myself the boss. Even the server thought that was a bit hasty.”

ANIMATION CUES

1. Question 1: requesting the admin role is not enough.
2. → Question 2: keeping the same email does not recover the account.
3. → Question 3: keeping the same sub finds the same account.
4. → Conclude: we need the admin account’s exact ID.

BACKGROUND

Several agents verified the behaviour: same sub, same member; new sub with an existing email, new member. The role/isAdmin fields do not change the role. The investigation also ruled out many account-modification and injection approaches, but the audience does not need to memorise the list. The sub ID links the statement to the stored member.

LIKELY QUESTIONS

Why not invent an ID? It would generally identify a new regular-user account. The goal is to find the existing admin record, not create an account with a convincing name. Attempts with common numbers or words did not find the right account.

SOURCES

ID_PLEASE · JOURNAL 18:39, 20:12, 20:25 · WRITEUP, SECTION 3

LIMITATIONS AND PROVENANCE

The results shown come from the documented tests. They do not establish that no other attack could work. Member IDs obtained before database resets could have belonged to other participants.

SLIDE 10 / 46 · 45–60 seconds

ID, please — finding the ID

WHAT TO SAY

“There was a clue on the homepage: the footer named Tech Corp Chandler and listed a Gmail address. The agent used a test Google session to look up that email through People API. The response returned the account’s Google ID, also called its Gaia ID. In this challenge, that is the expected sub. A statement identifying the real admin account can now be fabricated.”

ANIMATION CUES

1. Show the clue on the homepage.
2. → Describe the lookup using a test Google session.
3. → Reveal the abbreviated ID.
4. → Summarise the change of approach.

BACKGROUND

The solve journal and write-up document the lookup of techcorpchandler@gmail.com and the ID 103465053501192984574. The script claude/gaia_lookup.py preserves the lookup logic. No cookie, authentication header, or session secret is included in the presentation. The abbreviated ID on the slide is just a visual reference.

LIKELY QUESTIONS

Is the ID a password? No: it is an identifier. The weakness appears when the site accepts that ID in a statement without verifying its origin. The presentation does not claim that any Google email address can be resolved: it describes the lookup that worked for this CTF account.

SOURCES

ID_PLEASE · JOURNAL 20:35, 21:25 · WRITEUP, SECTIONS 4–5

LIMITATIONS AND PROVENANCE

The journal and write-up document the lookup result. The script is saved, but the cookies are outside the repository. An HTTP 401 observed earlier does not by itself prove that an account exists or uses social login; the final success confirms the targeted account in this case.

ID, please — an impersonated identity

WHAT TO SAY

“With the real ID, the fabricated statement finds member number 1, who is an administrator. The resulting session can then read the flag at /api/flag. The journal records the solve at 21:25. The lesson fits in one sentence: knowing an account’s ID is not proof that you own it. The site needed to verify the statement, not just read the ID it contained.”

ANIMATION CUES

1. Present the final chain.
2. → Reveal the results recorded in the write-up.
3. → Show the flag and the lesson.

BACKGROUND

The final payload carries the real sub and the account email. The token still does not supply the role: it comes from the existing administrator record. The write-up transcribes socialLogin with memberId=1 and role=admin, /api/me showing the admin member, then /api/flag with the flag. It corrects the earlier assumption about /api/admin/flag: that route returns 404 even for an admin.

LIKELY QUESTIONS

Who did what? The write-up credits several agents (pi, Codex, agy, qwen) with reconnaissance and ruling out approaches, and Claude with the breakthrough and final solve. Why so many dead ends? Authentication was weak, but the exact sub still had to be found; changing the role or email did not select the right account.

SOURCES

ID_PLEASE · JOURNAL 21:25 · WRITEUP, SECTIONS 6–8

LIMITATIONS AND PROVENANCE

Available evidence: the solve entry and the response transcribed in WRITEUP.md. No screenshot or separate final raw output was found in the inspected files. The panel on this slide transcribes the write-up; it is not a screenshot of the service. No exploit was rerun.

Universal translator — recovering a dictionary

WHAT TO SAY

“We receive a program for an unknown machine. It consists of small codes, but we no longer know which action each code represents. The machine uses a stack: it puts values on it, copies them, moves them or performs calculations. Fortunately, we also have four examples and the outputs they should produce. Those examples will let us rebuild the dictionary.”

Optional joke: “It knew exactly what it was saying. Our fluency in stack machine was the problem.”

ANIMATION CUES

1. Show the codes whose meaning is missing.
2. → Explain the stack in one sentence.
3. → Show the four examples: they serve as tests.

BACKGROUND

The archive contains two READMEs, four binary samples and transmission.bin, a 176-byte program. Codes 0x00 through 0x07 are a permutation of eight stack operations; codes 0x10 through 0x19 are a permutation of ten arithmetic operations. Other bytes push a literal value onto the stack. A stack is read from its top, the most recently added value. The boxes are a diagram, not a state taken from the samples.

LIKELY QUESTIONS

Why not try instructions at random? There is a finite set of possible dictionaries. Known outputs let us automatically reject the ones that do not fit. The challenge is to recover the machine’s language, then interpret the transmission.

SOURCES

UNIVERSAL_TRANSLATOR · JOURNAL 18:48–18:51 · ARCHIVE README

LIMITATIONS AND PROVENANCE

challenge.md is an empty template. The journal preserves the statement supplied by the user and the rules read from the README.

Universal translator — the stack and the test

WHAT TO SAY

“Let’s take a tiny invented example to see how this works. We pushed 7, then 3, so 3 is on top. We want to identify an instruction that runs before we print the top. If it copies, we get 3, 3, 7: the output is 3. If it swaps the two values, 7 moves to the top: the output is 7. If the example tells us the output must be 3, we can rule out ‘swap’. This is the principle the agent automates on the real programs.”

ANIMATION CUES

1. Read the stack from the top: 3, then 7.
2. → If the instruction copies, the output would be 3.
3. → If it swaps, the output would be 7.
4. → Reveal the expected answer, 3: swap is ruled out.

BACKGROUND

The top of the stack is drawn at the top. Copy duplicates the latest value; swap exchanges the two latest values. No actual hexadecimal opcode is assigned to the unknown instruction in this small example. The stack and expected output were chosen to explain the mechanism, not taken from the challenge samples. In the recovered mapping, DUP is 0x03 and SWAP is 0x06.

LIKELY QUESTIONS

Is this test enough to recover the whole machine? No. The consistent hypothesis has only passed one test. Other operations might produce the same output in a particular case. The solve uses all the supplied examples to distinguish complete dictionaries.

SOURCES

TEACHING DIAGRAM · STACK RULES DESCRIBED IN THE README

LIMITATIONS AND PROVENANCE

The numbers 7 and 3 and the expected output 3 form an invented teaching example, explicitly labelled on the slide. This is not a trace of the solve.

Universal translator — the real tests

WHAT TO SAY

“Back to the real challenge. The agent starts with stack operations: copy, swap, drop, emit… The first two examples reveal how they map to the codes. Next come the arithmetic operations. One example still leaves 346 possibilities under the correct interpretation of numbers. The last example leaves just one. The dictionary can finally be used to read the unknown transmission.”

ANIMATION CUES

1. Two groups of instructions are solved in sequence.
2. → Examples 1 and 2 determine the stack operations.
3. → Examples 3 and 4 determine the calculations: 346, then 1.
4. → The verified dictionary is used to read the transmission.

BACKGROUND

The solve uses exhaustive search. stack_candidates.py tests the 8! permutations of stack operations against examples 01 and 02. arithmetic_candidates.c then tests the 10! arithmetic permutations. Example 03 leaves 428 unsigned candidates or 346 signed candidates. After example 04, there are no unsigned candidates and exactly one signed candidate. The correct model uses signed 64-bit integers for MAX, MIN and right shift. Z3 was installed in the environment, but the saved solvers use exhaustive enumeration.

LIKELY QUESTIONS

Why mention signed numbers? A value can represent a negative number. That changes comparisons and some shifts. The agent tried both conventions; the known outputs confirmed which one worked. Do not describe 346 as the total number of initial possibilities.

SOURCES

UNIVERSAL_TRANSLATOR · JOURNAL 18:51–18:53 · CODEX/SOLVER-OUTPUT.TXT

LIMITATIONS AND PROVENANCE

Teaching diagrams based on the files. Animation timings do not represent the actual time taken to solve the challenges.

Universal translator — reading the transmission

WHAT TO SAY

“We now have the dictionary. For example, 03 means copy the top of the stack, 04 means emit a value, and 19 means add. The agent uses this reconstructed machine to read transmission.bin. It produces bytes that give us the flag when converted to text. The evidence is the saved solver output.”

ANIMATION CUES

1. Introduce the challenge.
2. → Reveal the next idea.
3. → Close with the evidence or the lesson.

BACKGROUND

The complete stack mapping is 00=ROT, 01=TUCK, 02=OVER, 03=DUP, 04=EMIT, 05=NIP, 06=SWAP, 07=DROP. The arithmetic mapping is 10=RSHIFT, 11=XOR, 12=LSHIFT, 13=*, 14=AND, 15=MIN, 16=MAX, 17=-, 18=OR, 19=+. Codes are hexadecimal. The transmission produces bytes 42 41 32 36 7b… which become BA26{0n3_st4ck_f0rth_4ll} in ASCII. The final scripts are codex/stack_candidates.py, codex/generate_data.py and codex/arithmetic_candidates.c.

LIKELY QUESTIONS

Did the agent contact a server? No target server is specified in this challenge’s archives. The documented work was local, using the supplied files. This presentation does not rerun the solver.

SOURCES

UNIVERSAL_TRANSLATOR · CODEX/SOLVER-OUTPUT.TXT · JOURNAL 18:53

LIMITATIONS AND PROVENANCE

The screenshot was annotated after the event. The mapping and decoded bytes are also recorded in the saved text output.

Silent beacon — the program and the decoy

WHAT TO SAY

“We receive a copy of a beacon’s memory. Its program asks for an activation code of eight hexadecimal characters. If the code is correct, it decodes a record. Searching the file for readable text also turns up a ready-made flag. It is a decoy. Our goal is to understand the code check, then follow the real decoding process.”

Optional joke: “All it needed was a sign saying: definitely not a trap.”

ANIMATION CUES

1. Introduce the memory dump.
2. → Introduce the code check.
3. → Show the plaintext decoy.
4. → Explain what would actually count as evidence.

BACKGROUND

The beacon_flash.bin dump is identified as ESP32-C6 firmware. The supplied program contains both the activation check and the decoder. The fake flag BA26{n0t_th15_0n3_k33p_d1gg1ng} appears in plaintext at offset 0x19d00. The reconstruction shows that the program reads only its first character for a boot message; the path that prints UNSEALED uses a different data region.

LIKELY QUESTIONS

Was the fake flag submitted? The archives do not say. Do not claim that it was. Did we have the physical device? The sources document a dump and an offline solver, not activation on a device.

SOURCES

SILENT_BEACON · JOURNAL 18:45, 18:49 · BACKFILL 20:57

LIMITATIONS AND PROVENANCE

The full challenge statement, exact disassembly commands and any actual activation are not preserved. The memory diagram is explanatory, not drawn to scale.

SLIDE 17 / 46 · 45–60 seconds

Silent beacon — the activation check

WHAT TO SAY

“The entire check is in the program. It takes two readable strings from the dump and calculates two numbers: fingerprints. It then combines them using a fixed recipe. The result is B80FEF82. That is the number it compares with the supplied code. The agent did not try every code: it repeated the calculation it had just read.”

ANIMATION CUES

1. The inputs to the calculation are the two strings in the dump.
2. → The program turns them into two fingerprints.
3. → It combines them to obtain the expected code.
4. → The check compares the supplied code with this answer.

BACKGROUND

The strings are LilyGO T7-C6 and BA26-BEACON-01. The check applies 32-bit FNV-1a to each. The saved hashes are 56ea3ff6 for LilyGO T7-C6 and cd1014a9 for BA26-BEACON-01. The expected code is rotl32(56ea3ff6,7) XOR cd1014a9 = B80FEF82. These values come from the saved output, not a new calculation for the presentation. Rotation moves the bits around in a loop; XOR combines the bits. Both technical terms are optional when speaking.

LIKELY QUESTIONS

What is a fingerprint? Here, a number calculated deterministically from a string. The important point is not to remember FNV, but that both the text and the recipe are known. An initial attempt reversed the two hashes and produced DEE06B10, so their order is essential.

SOURCES

SILENT_BEACON · CODEX/SOLVE.PY · CLAUDE/OUT-03-SOLVE.TXT

LIMITATIONS AND PROVENANCE

The displayed comparison illustrates the analysed logic. The archives do not prove that a code was entered on a device or service. The initial mistake is documented, but its output was not preserved.

SLIDE 18 / 46 · 45–60 seconds

Silent beacon — two separate calculations

WHAT TO SAY

“We need to distinguish two things. The first calculation checks the activation code. The second uses the same fingerprints to generate a sequence of numbers. We combine that sequence with the encrypted bytes in the dump, one byte at a time, and the text becomes readable. The expected code is therefore not simply a key applied to the file: the solver also recreates this second recipe.”

ANIMATION CUES

1. The two fingerprints are used by the check we already explained.
2. → They also generate a sequence for decoding.
3. → Combining this sequence with the 31 bytes reveals the text.
4. → The solver recreates both steps.

BACKGROUND

The decoder uses the two hashes as the initial 64-bit state of a xorshift generator, with shifts of 13, 7 and 17. The hash of LilyGO T7-C6 is the high word; the hash of BA26-BEACON-01 is the low word. At each step, the program produces a new state and XORs its low byte with one of the 31 bytes stored at address 0x42039d40. The dots are symbolic: they do not display calculated stream values.

LIKELY QUESTIONS

Why is the correct code not enough to explain the flag? It explains the access check, but we also need to understand how the content is transformed to account for the recovered text. Here, all the necessary data and operations are present in the dump, so the solver can reproduce them offline.

SOURCES

SILENT_BEACON · CODEX/SOLVE.PY · JOURNAL 18:49, BACKFILL 20:57

LIMITATIONS AND PROVENANCE

The diagram distinguishes two actual uses of the hashes; it is not an execution capture. No decoder was rerun to prepare this slide.

Silent beacon — evidence from decoding

WHAT TO SAY

“The solver recovers the expected code and decodes the record. This time, the flag really comes from the program’s success path, and it is recorded in the solver output. We can explain how it was obtained. That is what distinguishes it from the fake flag found in plaintext: the analysis connects the result to the actual challenge mechanism.”

ANIMATION CUES

1. Introduce the challenge.
2. → Reveal the next idea.
3. → Close with the evidence or the lesson.

BACKGROUND

codex/solve.py reads the dump, extracts the identity strings, calculates the hashes and code, then reproduces the xorshift and XOR operations on the 31 bytes. The saved output reports activation code=B80FEF82 and record=BA26{x0rsh1ft_s34ls_th3_b34c0n}. The detailed technical steps are in Codex’s original journal and in the reconstruction based on the saved disassembly.

LIKELY QUESTIONS

Does this prove a physical device was unlocked? No: the documented evidence is the offline calculation and decoding. The journal contains no record of the code being entered on a device or service. What we can say is that the solver reproduces the analysed logic and recovers the recorded result.

SOURCES

SILENT_BEACON · CLAUDE/OUT-03-SOLVE.TXT · JOURNAL 18:49

LIMITATIONS AND PROVENANCE

The output screenshot was made during the backfill from an offline reproduction. It matches the activation code and flag recorded by Codex during the solve.

Wrong path — leaving the directory

WHAT TO SAY

“FileVault lets us upload and read files. The server is supposed to stay inside our account’s directory. Look at the documented request: we ask for ‘dot dot slash twice, then challenge.db’. The server appends it to our account’s path. The first ‘../’ moves from our account up to uploads; the second moves up to app. The target becomes /app/challenge.db: the account database. Another route, /download, did block this escape. The agent used /raw.”

Optional joke: “We had access to our drawer. Two dots and a slash later, we were touring the whole cabinet.”

ANIMATION CUES

1. Read the request and identify the allowed directory.
2. → Show the constructed path and explain ../.
3. → Show the final file outside the directory.
4. → Compare the safe route and the vulnerable route.

BACKGROUND

The request GET /raw?filename=../../challenge.db and the root /app/uploads/<username>/ appear in JOURNAL.md, in the App and Exploitation sections. The journal describes open(os.path.join(UPLOAD_FOLDER,user,filename)) without sanitisation for /raw and /view, versus send_from_directory for /download. The displayed path resolution illustrates what ../ means; it is not a newly retrieved HTTP response. Before reading the database, ../../app.py had confirmed its location. /etc/passwd was used to prove traversal. The ....// attempt failed; ../ and its encoded form worked according to the notes.

LIKELY QUESTIONS

Why does os.path.join not protect this? It constructs a path; the documented mechanism does not then enforce that the final file remains under the allowed directory. This solve does not show that every route in the app was vulnerable: the contrast between /download and /raw is essential.

SOURCES

WRONG_PATH/JOURNAL.MD · APP, EXPLOITATION, SOLVED 20:07

LIMITATIONS AND PROVENANCE

No screenshot of the web interface or raw HTTP request/response was preserved. The paths and request are transcribed from the journal; the account name is replaced by a generic placeholder.

Wrong path — the fake flag

WHAT TO SAY

“The database contains a password that looks like a flag. The agent initially thinks it is done, then corrects that conclusion: it was a trap. The real administrator credentials are in the same database. After logging in as admin, it finds a file saying that the real flag is hidden in a maze.”

ANIMATION CUES

1. Introduce the challenge.
2. → Reveal the next idea.
3. → Close with the evidence or the lesson.

BACKGROUND

The journal explicitly corrects the earlier claim: the BA26-dftwp-… string is a decoy, not the final result. Reading /proc/self/environ revealed SECRET_KEY and confirmed the credentials’ location. The admin account in challenge.db granted access to /dashboard, which listed flag.txt and flag.maze. The final solve uses those credentials; do not invent session forgery using SECRET_KEY.

LIKELY QUESTIONS

How do we know the first flag was fake? The journal explicitly corrects it, and the real result is preserved in claude/out-01.txt. The presentation follows that correction, not the outdated initial summary.

SOURCES

WRONG_PATH · CLAUDE · JOURNAL, CORRECTION AND RECONSTRUCTION

LIMITATIONS AND PROVENANCE

The journal reconstructs the broad sequence after the event. There are no HTTP traces that would precisely timestamp every action.

Wrong path — understanding BFS

WHAT TO SAY

“The maze has more than forty thousand cells: reading it by eye would be tedious. The program uses breadth-first search. It keeps a queue of cells to explore. When it visits a cell, it adds accessible neighbours it has not seen yet, remembering where they came from. Once it finds the exit, it follows those links back to reconstruct the path. Only letters on that path are kept.”

ANIMATION CUES

1. Read the five algorithm lines without explaining Python.
2. → Explain why every cell stores a predecessor.
3. → Trace backwards from the exit to the entrance.
4. → Keep letters on the route, not on the branches.

BACKGROUND

claude/solve.py reads the grid, locates the two > characters, then initialises prev={start:None} and q=deque([start]). It tests the four neighbours; a cell is allowed if it is inside the grid and is not a #. The prev dictionary marks visited cells and links them to their predecessors. The solver stops when it reaches the exit, reconstructs and reverses the path, then removes walls, spaces and gates before joining the letters. The slide’s pseudocode is a simplified description of these operations.

LIKELY QUESTIONS

Why store the predecessor? A list of visited cells does not directly give the final route: it includes unnecessary branches too. The links let us start at the exit and recover the exact sequence of cells. No formal graph theory is needed to follow this idea.

SOURCES

WRONG_PATH/CLAUDE/SOLVE.PY · CELL QUEUE, PREV, RECONSTRUCTION

LIMITATIONS AND PROVENANCE

The pseudocode is a teaching translation of the saved solver, not an execution trace. The search was not rerun to produce the presentation.

Wrong path — the maze

WHAT TO SAY

“The file is a text maze: walls, corridors and letters. The solver explores passages from the entrance to the exit. It keeps that path, then reads the characters along it. Together, they form the real flag. The green drawing shows the recovered path rendered from the saved file.”

ANIMATION CUES

1. Introduce the challenge.
2. → Reveal the next idea.
3. → Close with the evidence or the lesson.

BACKGROUND

The maze is 201 × 201. Its two gates are at (row 1, column 0) and (row 199, column 200). claude/solve.py uses breadth-first search, or BFS, then traces the predecessors to reconstruct the path. The saved output reports a length of 2 783 positions. The full flag is BA26{tHis_Is_a_rEally_loNG_flag_ThAt_is_Very_DiFFicult_to_gUess_bUT_you_cAn_find_It_in_tHE_mAze}.

LIKELY QUESTIONS

What is breadth-first search? It visits neighbouring cells first, then their neighbours, layer by layer. The audience can remember ‘explore the corridors while keeping track of where you came from’. The green path represents the saved result, not an animation of the original execution.

SOURCES

WRONG_PATH/CLAUDE/SOLVE.PY · OUT-01.TXT · RENDER_MAZE.PY

LIMITATIONS AND PROVENANCE

The drawing was created after the event from the local maze file. The string on the slide is deliberately shortened; the evidence and notes give the full result.

Peak privilege — the challenge

WHAT TO SAY

“This system grants permissions based on a badge image. It classifies each image as employee, invalid or administrator. To get the flag, its ADMIN score must reach 99.5%. We do not have the model, but the service returns its scores. That feedback will guide the changes to the image.”

ANIMATION CUES

1. Introduce the challenge.
2. → Reveal the next idea.
3. → Close with the evidence or the lesson.

BACKGROUND

The input is a 256 × 176 RGB image. The known preprocessing crops a 96 × 96 square, converts it to grayscale, then averages 3 × 3 blocks to produce a 32 × 32 grid. The agent can therefore control 1 024 values through blocks of pixels. The model weights are not provided. The initial employee badge has an EMPLOYEE score close to 1 and an ADMIN score of roughly 2.3×10^{-23}.

LIKELY QUESTIONS

Is this identity card recognition? The archives describe an artificial CTF scanner called NeuralBadge with three classes. Do not generalise this result to a real product. The returned score is a model output, not a guarantee of real-world probability.

SOURCES

PEAK_PRIVILEGE/JOURNAL.MD · SETUP · CLAUDE/EXTRACTED/PLAYER/PREPROCESS.PY

LIMITATIONS AND PROVENANCE

The description comes from the journal because challenge.md is an empty template. The employee image is a supplied file; the scanner drawing is an illustration.

Peak privilege — controlling the input

WHAT TO SAY

“Before looking for a winning badge, the agent reads the image preprocessing code. The model does not see the whole card: just this square. It is converted to grayscale, then each three-by-three block of pixels becomes one average value. That produces a 32 by 32 grid. If we put nine identical pixels in a block, we choose the value of the corresponding cell. The agent can therefore control every input to the model.”

ANIMATION CUES

1. Point to the square read from the badge.
2. → Show what gets cropped, then averaged.
3. → Explain how nine identical pixels control one cell.
4. → Conclude: we control the entire useful input.

BACKGROUND

preprocess.py specifies BADGE_SIZE=(256,176), PATTERN_BOX=(32,48,128,144) and a luminance conversion of 0.299R+0.587G+0.114B, followed by reshape(32,3,32,3).mean(axis=(1,3)). Setting R=G=B=k in a block gives k/255, or 256 controllable levels per cell. claude/work/badge.py performs the inverse operation to produce a PNG: grid_to_png enlarges the grid and places it in the square. The journal reports a maximum difference of 1.2×10^{-7} in the round-trip test. The outline and crop shown use the exact coordinates from the specification; the grid on the right is schematic.

LIKELY QUESTIONS

Why does this step matter? Changing text or pixels outside the square would be pointless: they are discarded before reaching the model. Having the right image dimensions is not enough; we need to understand what actually reaches the classifier. Control is quantised to 256 levels, not infinitely precise real numbers.

SOURCES

PEAK_PRIVILEGE/CLAUDE/EXTRACTED/PLAYER/PREPROCESS.PY · CLAUDE/WORK/BADGE.PY

LIMITATIONS AND PROVENANCE

The image is the supplied badge. The crop and outline visually explain the supplied code; the 32 × 32 grid is schematic and does not claim to show its 1 024 measured values.

Peak privilege — warmer, colder

WHAT TO SAY

“The agent starts from the real employee badge because random noise mostly produced INVALID. Each round, it chooses twenty variations. For each one, it tests the badge changed in one direction and in the opposite direction: forty images in total. If one direction gets closer to ADMIN, the score difference points to a useful direction. It combines the twenty signals, changes the badge, and repeats. It is ‘warmer, colder’, measured and automated.”

ANIMATION CUES

1. Explain why we start from a recognised badge.
2. → Compare both directions of the same variation.
3. → Combine the directions and update the badge.
4. → Repeat the loop until acceptance.

BACKGROUND

The nes.py solver constructs x+sigma·eps[k] and x−sigma·eps[k] for each of the 20 Gaussian perturbations. Its objective is log(pADMIN) − log(max(pEMPLOYEE,pINVALID)). The difference between the two objective values weights the perturbation; their average estimates a gradient. It normalises this gradient, takes a step of size lr, then clips x to [0,1]. Conversion to PNG quantises it to 256 levels. The best ADMIN score and its grid are saved. The journal reports two phases: sigma=0.06 and lr=0.08, then lr=0.28. The script also checks the current badge outside the batch of variations.

LIKELY QUESTIONS

Does it simply pick the best badge from the batch? No: it estimates a direction from the pairs, then builds a new image. The twenty pairs help combine noisy signals. ‘More ADMIN’ simplifies the comparison: the objective also accounts for the strongest competing class. Not every step is necessarily better than the previous one.

SOURCES

PEAK_PRIVILEGE/CLAUDE/WORK/NES.PY · SCORE, ±EPS PAIRS, GRADIENT, UPDATE

LIMITATIONS AND PROVENANCE

The two variants are shown by their roles, with no invented image or score. The journal keeps a summary of the attempts, not the complete series of HTTP responses.

Peak privilege — the accepted badge

WHAT TO SAY

“The final badge looks strange. Yet the journal reports an ADMIN score of 99.6%, above the 99.5% threshold, followed by access granted and the flag. The winning image was saved. The lesson from this challenge is that the model gives the attacker a measurement that helps them build an accepted input.”

Optional joke: “Clearly, the model and HR have very different standards for badge photos.”

ANIMATION CUES

1. Introduce the challenge.
2. → Reveal the next idea.
3. → Close with the evidence or the lesson.

BACKGROUND

The journal reports roughly 113 batch calls of 40 images across the entire solve; these counts remain approximate. The second phase reaches the threshold after around 53 iterations. The badge is claude/work/admin_badge.png, with the winning grid in best_grid.npy. The final solver is nes.py. The result and flag are recorded in the SOLVED section.

LIKELY QUESTIONS

Does the screenshot alone prove the score? No: it shows the winning badge. The score, access granted and flag come from the journal. No raw service response log was kept; the presentation explicitly states this in the provenance notes.

SOURCES

PEAK_PRIVILEGE/JOURNAL.MD · SOLVED · CLAUDE/WORK/ADMIN_BADGE.PNG, BEST_GRID.NPY

LIMITATIONS AND PROVENANCE

The original notes are undated; the reconstruction's times come from file timestamps. The score of 0.996 is reported in the journal, with no raw HTTP response preserved.

Rucher — the two files

WHAT TO SAY

“We have ten seconds of radio signal and the program from a scale. There are three layers to the solve. First, turn the waves into bytes: radio decoding. Next, recover the Zigbee key to read the reports: network encryption. Finally, those reports contain fragments still encrypted by the application. The firmware explains this last rule, which is tied to the scale's identity.”

Optional joke: “For a challenge called Rucher, the real buzz came from the scale.”

ANIMATION CUES

1. Distinguish the radio signal from the embedded program.
2. → Show network encryption as a second layer.
3. → Show the application's own protection.
4. → Explain that each step has its own verification check.

BACKGROUND

The 80 MB SigMF capture contains ci8 samples at 4 MS/s, centred on 2 475 MHz. The journal identifies IEEE 802.15.4 channel 25 and Zigbee. The balance_flash.bin dump is partial ESP32-C6 firmware. Its strings name Zigbee and the default key; its NVS area is blank, so it does not contain the provisioned network key of the observed scale. The journal attributes the entire solve to Claude. The slide's three layers group demodulation, Zigbee security and application-level RC4 protection; they do not claim to represent every official protocol sublayer.

LIKELY QUESTIONS

Why are both files needed? The radio recording provides the pairing exchanges, fragments and device identity. The firmware explains the protocol and how the final key is built. There is no ready-to-read flag in this factory dump.

SOURCES

RUCHER/JOURNAL.MD · SECTIONS 0–4 · RUCHER.SIGMF-META · BALANCE_FLASH.BIN

LIMITATIONS AND PROVENANCE

The diagram is a teaching overview of the solve steps. The timeline was reconstructed from the files; the dump is truncated and does not contain complete partitions beyond its size.

SLIDE 29 / 46 · 45–60 seconds

Rucher — from waves to packets

WHAT TO SAY

“At first, all we have is recorded radio waves. The agent identifies Zigbee and writes a decoder. It compares the signal with the expected patterns to recover the transmitted numbers. The packets include an error check: a useful way to verify the decoding. The result: all 42 packets pass that check. We can now analyse the messages.”

ANIMATION CUES

1. Introduce the challenge.
2. → Reveal the next idea.
3. → Close with the evidence or the lesson.

BACKGROUND

The decode154.py and extract_all.py scripts detect bursts and use a complex matched filter for the O-QPSK/DSSS patterns. ×4 oversampling improves precision. The CRC-16 FCS confirms the modulation conventions and bytes. The result is saved in claude/rucher.pcap, with 42 decoded bursts and valid FCS values. The capture includes association followed by periodic reports.

LIKELY QUESTIONS

Does the error check decrypt the message? No. It confirms that the radio bits were read correctly. The content can still be encrypted. That is why decoding and decryption are separate steps.

SOURCES

RUCHER/CLAUDE/DECODE154.PY · EXTRACT_ALL.PY · JOURNAL.MD, SECTION 2

LIMITATIONS AND PROVENANCE

The drawn waveform illustrates the signal; it is not a measured plot. The 42/42 count comes from the journal and the preserved packet file.

Rucher — the network key

WHAT TO SAY

“The scale receives its network key when it joins the gateway. That exchange is in our capture. The key is not sent in plaintext: the message is protected by a transport key derived from a known default key. The agent applies that derivation, opens the Transport-Key message and verifies its authenticity check. It now has the network key. This opens the reports and reveals six fragments, which still have application-level protection.”

ANIMATION CUES

1. Key distribution was recorded.
2. → Derive the transport key from the default key.
3. → Open the message and verify its authenticity.
4. → Use the network key to read the six fragments.

BACKGROUND

The APS Transport-Key message is protected with a transport key derived from ZigBeeAlliance09 using Zigbee AES-MMO hashing with byte 0x00. Do not say that the default key is applied directly to the packet without this derivation. AES-CCM* decrypts and verifies the MIC. The recovered network key is b8d8e46f82f99446568d2d30e9512c16. The verified NWK reports contain six fragments indexed from 0 to 5, totalling 46 bytes.

LIKELY QUESTIONS

Is the key sent in plaintext? No: it is encrypted, but its protection derives from a known default key. Why capture the pairing? In this solve, the preserved exchange provides the network key needed to read the reports.

SOURCES

RUCHER/CLAUDE/SOLVE_FINAL.PY · ZB_KEYED_HASH, CCM_STAR, TRANSPORT-KEY

LIMITATIONS AND PROVENANCE

The derivation and encryption algorithms are named in the notes. The slide explains how the keys depend on each other without claiming that a plaintext key was transmitted over radio.

Rucher — the device-specific layer

WHAT TO SAY

“The firmware reveals how the final key is built. It concatenates the scale's radio identifier, the word ‘propolis’ and the fragment number. The identifier, an eight-byte EUI-64 address here, is visible in the capture. It means two scales use different keys, but it is not a secret to the observer. The agent therefore rebuilds each key, applies RC4 normally and assembles the six pieces. This is not a mathematical break of RC4.”

ANIMATION CUES

1. Introduce the challenge.
2. → Reveal the next idea.
3. → Close with the evidence or the lesson.

BACKGROUND

The identified algorithm is RC4, reinitialised for each fragment. The exact key is the EUI-64 in little-endian order, followed by the bytes of ‘propolis’, then one index byte. The scale's EUI-64 is 40:4c:ca:ff:fe:46:87:c4, or c48746feffca4c40 in little-endian. The final solver, claude/solve_final.py, takes the pcap, recovers the network key, decrypts the reports and the RC4 layer, then concatenates the fragments. The preserved output confirms the displayed flag.

LIKELY QUESTIONS

Is the scale's identity secret? It is visible in the capture. Here, it makes the encryption device-specific, but it is not enough to keep the content secret once the mechanism is understood. Why does the firmware alone not provide the flag? It lacks this scale's messages and its observed identity.

SOURCES

RUCHER/CLAUDE/SOLVE_FINAL.PY · PER-FRAGMENT KEY · OUT-01-SOLVE.TXT

LIMITATIONS AND PROVENANCE

The result shown comes from an offline reproduction preserved during the backfill. No solver was rerun for this presentation. The earliest tool logs and the initial demodulation attempt were not preserved.

SLIDE 32 / 46 · 60–75 seconds

Driver radio — the message and its proof

WHAT TO SAY

“The challenge asks us to send exactly ‘Box Box Box!’ to all 22 drivers. But the program also requires a valid signature for the selected driver. The private key creates that signature; the public key verifies it. The public keys are in the program we received. So we are missing the secret needed to sign. A signature does not hide the message: here, it lets the program check that the order is authorized.”

ANIMATION CUES

1. Set the goal: one order accepted by 22 drivers.
2. → Explain the private key and signature.
3. → Explain verification with the public key.
4. → Name the missing secret.

BACKGROUND

The supplied binary is an x86-64 ELF with symbols and debug information. The relevant functions are menu_spy, menu_send, ecdsa_sign and ecdsa_verify. menu_send verifies the ECDSA P-256 signature and marks the driver if the text is exactly “Box Box Box!”. The program displays the flag when all 22 markers are set. The PILOT_QX and PILOT_QY arrays contain the coordinates of the 22 public keys.

LIKELY QUESTIONS

Why is reading a public key not already a vulnerability? It is meant to be known. We should be able to verify a signature without recovering the private key. A separate weakness breaks that separation here.

SOURCES

driver_radio/JOURNAL.md: entries at 20:00 and 20:05; claude/send_exploit.py.

LIMITATIONS AND PROVENANCE

challenge.md is also an empty template in driver_radio. The slide's context comes from the journal and the documented binary logic. The radio drawing is a teaching illustration, not a screenshot of the program.

SLIDE 33 / 46 · 60–75 seconds

Driver radio — a tiny random space

WHAT TO SAY

“The signature uses another, temporary secret: the nonce. This is not the private key. In this program, it has only 255 possible values. The Spy menu reveals a signature even though the message is encrypted. Part of that signature depends on the nonce. The agent can therefore try every value and compare the results. This search recovers the nonce; on its own, it does not yet reveal the driver's key.”

ANIMATION CUES

1. Distinguish the nonce from the private key.
2. → The signature gives us a value to compare.
3. → Every candidate can be tried.
4. → State what is still missing.

BACKGROUND

The journal gives k = (rand() % 255) + 1 and r = (k·G).x mod n. G is the curve's base point; n is its order. The solver builds a dictionary of r values for the 255 possible k values. This computation is done once, then r is used for lookup. The message is shown encrypted with AES-CTR, but the signature (r,s) remains visible. The vulnerability described here concerns ECDSA nonce generation.

LIKELY QUESTIONS

Is this a search through 255 private keys? No. The grid represents only the 255 nonces permitted by this implementation. Recovering the private key still requires the signature and the message. The P-256 curve obviously has far more than 255 private keys. This slide does not simulate a winning number.

SOURCES

driver_radio/JOURNAL.md: 20:05 and 20:15; claude/recover.py, the rk dictionary; screenshots/claude-02-nonce-vuln.annotated.png.

LIMITATIONS AND PROVENANCE

The saved decompilation file is empty. The vulnerable section is documented in the journal and the annotated screenshot; the complete disassembly was not kept. This screenshot is archival evidence, not code extracted again for this presentation.

SLIDE 34 / 46 · 60–75 seconds

Driver radio — the message breakthrough

WHAT TO SAY

“The nonce has been recovered, but the signed text is still hidden. The first idea was to reproduce the randomness from the clock. The breakthrough came from looking at the messages: they follow just a few templates. An engine mode, a plan, a pit-stop lap… The variables have few possible values. So the agent enumerated the messages, about 800 per driver. Each message became a hypothesis to test.”

ANIMATION CUES

1. Briefly explain the clock-based approach.
2. → Read one message template and its variables.
3. → Reveal the limited number of candidates.
4. → Summarize what the breakthrough simplifies.

BACKGROUND

The journal describes about 800 messages per driver, built from eight message types and small, deliberately widened value ranges. recover.py keeps extra candidates to cover these ranges. The file supports the formats shown. The driver's name is part of the message: “[pilote]” on the slide is a teaching abbreviation. The actual strings include the full name. In the template placeholders, lettre means letter, signe means sign, tour means lap and pneu means tyre.

LIKELY QUESTIONS

Did the agent break AES? The final solution does not need to: it enumerates plausible texts and checks the resulting private key against the known public key. Why abandon the clock? The new approach removes dependence on the seed, the time and the alignment of rand() calls.

SOURCES

driver_radio/JOURNAL.md: 20:07, 20:12, 20:15; claude/recover.py, the enum() function.

LIMITATIONS AND PROVENANCE

The planned local validation of the first approach was blocked by the absence of libcrypto.so.3 for x86-64. The examples shown are templates from the solver, not a transcript of a specific exchange with a driver.

SLIDE 35 / 46 · 60–75 seconds

Driver radio — calculate, then confirm

WHAT TO SAY

“We combine three things: the observed signature, the recovered nonce and a possible message. The mathematical relationship behind the signature lets us calculate a candidate private key. But we do not just trust it. We calculate its public key and check that it belongs to the driver. If it does not match, we try another message. The solver recovered and confirmed all 22 private keys this way.”

ANIMATION CUES

1. Bring the three pieces together.
2. → Calculate a candidate.
3. → Explain the test and its two outcomes.
4. → Announce all 22 confirmations.

BACKGROUND

Documented ECDSA equation: $s = k^{-1}(z + r \cdot d) \bmod n$. Multiply by k, subtract z, then multiply by the inverse of r to obtain $d = r^{-1}(s \cdot k - z) \bmod n$. $z = \text{SHA256}(\text{message})$. r^{-1} is the multiplicative inverse modulo n, not ordinary decimal division. d is the private integer, G the base point and Q the public key. The test $d \cdot G = Q$ verifies the candidate. In recover.py, this is eqpt(mulG(d), Q).

LIKELY QUESTIONS

Why guess the message? Without z, the calculation of d is not determined. Why collect several signatures? The journal records ten observations per driver, or 220 signatures; the solver works through the signatures until it finds a confirmed key. We should therefore not claim that the actual attack required only one observation per driver.

SOURCES

driver_radio/JOURNAL.md: 20:15–20:16; claude/recover.py, key recovery and comparison; claude/crypto.py.

LIMITATIONS AND PROVENANCE

This presentation uses the saved scripts and results. It does not recalculate keys or signatures. The private keys do not need to be displayed to explain or demonstrate the result.

SLIDE 36 / 46 · 60–75 seconds

Driver radio — 22 orders accepted

WHAT TO SAY

“With the recovered keys, the agent can sign ‘Box Box Box!’ for each of the 22 drivers. It verifies those signatures locally, then sends them over a new connection. The saved response shows all 22 accepted commands and the flag. The conclusion is specific: here, an easily guessed temporary number and predictable messages made it possible to recover the signing keys.”

Optional light aside: “Imagine the mechanics' faces: twenty-two drivers arrive at once!”

ANIMATION CUES

1. Announce the overall result.
2. → Show the archived screenshot.
3. → Recap the three main stages.
4. → Give the takeaway without generalizing to all signatures.

BACKGROUND

Original file chain: harvest.py → sigs.json → recover.py → priv.json → send_exploit.py → final_resp.txt. According to the journal, keys remain the same across connections, so the agent could recover them offline and then sign in a new connection. The recorded flag is BA26{T0t0_1ts_C4ll3d_4_M0t0r_R4c3_N0t_S3cur3_Crypt0}.

LIKELY QUESTIONS

Is this a general break of ECDSA? The archives show an exploit of this implementation: nonces from 1 to 255 and enumerable messages. Do not generalize the result to every implementation or every use of signatures.

SOURCES

driver_radio/JOURNAL.md: 20:19–20:20; claude/final_resp.txt; claude/send_exploit.py; screenshots/claude-01-flag.annotated.png.

LIMITATIONS AND PROVENANCE

The screenshot was annotated after the event. The result is corroborated by final_resp.txt and the solve journal. Animation timings do not represent the actual exploitation speed.

Tapo — the challenge

WHAT TO SAY

“A camera is also a small computer. Its memory holds programs, settings and files. Here is the hardware photographed during the work. The challenge is to recover a hidden message. The camera was watching the garden; for once, we are the ones taking a closer look at it.”

ANIMATION CUES

1. Introduce the challenge.
2. → Reveal the next idea.
3. → Close with the evidence or the lesson.

BACKGROUND

The source presentation identifies a Tapo C500 2.0 and an 8 MiB dump. Firmware is the device's embedded software. A dump is a raw copy of flash memory. The photograph was extracted from the supplied PowerPoint without changing the original.

LIKELY QUESTIONS

Is a dump a video? No: here, it is a copy of the memory containing the device's software and data.

SOURCES

TAPO · SUPPLIED POWERPOINT, SLIDES 1–4

LIMITATIONS AND PROVENANCE

Photograph supplied by the user. The dump itself was not attached to this request; no memory acquisition or code execution was performed for the presentation.

SLIDE 38 / 46 · 45–60 seconds

Tapo — the hardware and the dump

WHAT TO SAY

“The clip and wires connect the board to an adapter, which sits between it and the computer. The source presentation describes reading the memory with a CH347 in SPI mode and flashrom. The result is an eight-mebibyte file. Everything that follows is about understanding that copy. Plenty of cables, but just one attachment to analyze.”

ANIMATION CUES

1. Introduce the challenge.
2. → Reveal the next idea.
3. → Close with the evidence or the lesson.

BACKGROUND

SPI is a communication method used between electronic components. The adapter and setup are visible in the photographs. The CH347/SPI/flashrom readout is described in the PowerPoint notes. We do not infer a pinout or propose a hardware procedure from the photos.

LIKELY QUESTIONS

Why work on a copy? It lets us read and compare the bytes offline. That does not establish whether the hardware setup itself was safe or reproducible: neither was verified here.

SOURCES

TAPO · POWERPOINT PHOTOS AND NOTES, SLIDES 3–5

LIMITATIONS AND PROVENANCE

The four photos were supplied by the user. Their order on the slides is for teaching, not a timeline of the hardware work.

Tapo — compression versus encryption

WHAT TO SAY

“Searching memory for BA26 is a reasonable first instinct. The source presentation reports several unsuccessful searches. There are two layers to distinguish. Compression stores files in less space: we need to decompress them. Encryption then transforms the flag's contents: we need to recover the recipe and its inputs. A ZIP file and a safe are two different puzzles.”

ANIMATION CUES

1. Introduce the challenge.
2. → Reveal the next idea.
3. → Close with the evidence or the lesson.

BACKGROUND

The files are recovered from XZ blocks in a filesystem described as SquashFS. flag.enc contains 35 bytes. The six bytes shown are the first six bytes of the supplied file. XZ and ZIP are different formats; the analogy is only about compression.

LIKELY QUESTIONS

Does the absence of BA26 prove encryption? No. The later analysis of the files and ctf_unlock establishes the two layers.

SOURCES

TAPO · POWERPOINT SLIDES 7–10 · FLAG.ENC · SOLVE.PY

LIMITATIONS AND PROVENANCE

The initial searches are reported in the source presentation. No simulated terminal is presented as a real screenshot.

Tapo — the overlooked region

WHAT TO SAY

“The turning point was a region that the first extraction had missed. Imagine a book without its cover: you cannot read the title, but the table of contents and pages are still there. Here, the metadata still tells us where to find the files, so we can recover them directly. The book was poorly labeled, not empty.”

ANIMATION CUES

1. Introduce the challenge.
2. → Reveal the next idea.
3. → Close with the evidence or the lesson.

BACKGROUND

The script uses a base at 0x1c0000, decompresses two XZ metadata blocks at offsets 0x3a8236 and 0x3a887c, then a fragment table at 0x3aa18a. The fragment_file function reads inode and fragment information to extract the data. Clarification from version (2), slide 14: read_bytes reads the dump; xz decompresses the blocks; the tables describe the files and where their data is located. fragment_file finds the block, decompresses it if necessary, then slices out the file's bytes. The locations are already known and hardcoded in the script: it is specific to this dump and does not discover them automatically. unlock holds the extracted program; encrypted holds the encrypted message. The PPT reports an unrecognizable SquashFS superblock.

LIKELY QUESTIONS

Was the header deliberately sabotaged? The source presentation does not establish that. We can only say that it was not recognizable and that usable metadata remained.

SOURCES

TAPO · POWERPOINT SLIDES 9–10 · SOLVE.PY

LIMITATIONS AND PROVENANCE

The book analogy is for teaching. The offsets can be read in the script, but extraction was not rerun.

SLIDE 41 / 46 · 45–60 seconds

Tapo — the ingredients

WHAT TO SAY

“We find the encrypted message, a small program named ctf_unlock and a startup script that tells us how they fit together. That script uses the camera's identifier. In other words, the camera locked the safe but left the opening instructions in the same box. We still need to understand those instructions.”

ANIMATION CUES

1. Introduce the challenge.
2. → Reveal the next idea.
3. → Close with the evidence or the lesson.

BACKGROUND

The supplied ctf_unlock file is a statically linked, stripped 32-bit MIPS ELF executable. The PPT says the startup script reads devId, passes it to the program with flag.enc and writes the result to /tmp/flag.txt. solve.py recovers devId from the dump using a regular expression.

LIKELY QUESTIONS

Is the identifier an inaccessible secret key? In this challenge, it can be recovered from the dump. This observation does not establish a general vulnerability in all Tapo products.

SOURCES

TAPO · POWERPOINT SLIDES 11–12 · CTF_UNLOCK · FLAG.ENC · SOLVE.PY

LIMITATIONS AND PROVENANCE

The startup script and disassembly are described in the PowerPoint; they were not provided separately. The binary was not executed.

Tapo — the mask

WHAT TO SAY

“The program creates a sequence of numbers from the identifier, a short fixed string and a counter. That sequence serves as a mask. Encryption uses an operation called XOR: with the same mask, we can recover the original message. The agent translated the recipe into Python. No crystal ball needed: the ingredients were already in the file.”

ANIMATION CUES

1. Introduce the challenge.
2. → Reveal the next idea.
3. → Close with the evidence or the lesson.

BACKGROUND

Recipe visible in solve.py: SHA256(devId_ASCII || C500-CTF-v1 || LE32(compteur)), with the counter starting at zero. Each hash produces 32 bytes; two hashes cover the 35 encrypted bytes. XOR is applied byte by byte. devId is used directly as ASCII, without hexadecimal conversion. Clarifications from version (2), slide 15: re.search extracts the devId field; struct.pack encodes the counter as four little-endian bytes; digest produces 32 bytes per hash; join concatenates the blocks. The block count rounds up: 35 bytes require counters 0 and 1. zip pairs the encrypted bytes with the stream and stops at the shorter input, so only 35 bytes are used. The ^ symbol performs XOR, bytes rebuilds the result and decode converts it to text. The assert checks the BA26 format, not platform acceptance, which the user confirmed separately.

LIKELY QUESTIONS

Did we break SHA-256? No. The function is used in the normal direction to reconstruct a stream whose inputs are known. The script extracts ctf_unlock but does not execute it.

SOURCES

TAPO · SOLVE.PY · POWERPOINT SLIDE 13

LIMITATIONS AND PROVENANCE

The recipe was checked by reading the script. No decryption or extraction was rerun for these slides.

Tapo — XOR, without the magic

WHAT TO SAY

“Here is an operation we can explain with four bits. If the two bits are identical, XOR gives zero; otherwise, it gives one. Who can work out the result on the left? Pause for a few seconds. One, one, zero, zero. Now take that result and apply the same mask: we get the original message back. That is exactly the principle used to remove the mask from the encrypted file.”

ANIMATION CUES

1. Explain the rule and let the audience calculate.
2. → Reveal 1100.
3. → Apply the same mask again: 1010.
4. → Connect the calculation to file decryption.

BACKGROUND

The identity is (message XOR mask) XOR mask = message. The script applies it to bytes, each containing eight bits. The real mask comes from the two SHA-256 hashes on the previous slide.

LIKELY QUESTIONS

Is this inherently weak encryption? XOR is an operation; confidentiality depends, among other things, on the secrecy and use of the mask. Here, its ingredients can be recovered.

SOURCES

TAPO · POWERPOINT SLIDES 14–15 · THE ^ OPERATION IN SOLVE.PY

LIMITATIONS AND PROVENANCE

An invented mini-example, explicitly distinct from the real bytes. The animation waits for the speaker before revealing the answer.

Tapo — understanding the script

WHAT TO SAY

“The first line assembles the identifier, fixed label and counter. The second computes one block of the mask. Two blocks are enough here. The third pairs each encrypted message byte with its mask byte, then applies XOR. This code reproduces the MIPS program's logic; it does not execute that program on our computer.”

ANIMATION CUES

1. Connect key to devId and explain the counter.
2. → SHA-256 generates the mask.
3. → XOR recovers the text's bytes.
4. → Distinguish the recipe, archived result and submission.

BACKGROUND

The three expressions come from the supplied solve.py, with the sha256 arguments shortened to an ellipsis. struct.pack encodes an unsigned integer as four little-endian bytes. digest returns 32 bytes. The block count is (len(encrypted)+31)//32. zip stops after the 35 encrypted bytes. Earlier, fragment_file extracts ctf_unlock and flag.enc from the decompressed tables; re.search recovers devId. The offsets are fixed: this solver is specific to the dump. The assert checks the BA26{…} format, not server validation.

LIKELY QUESTIONS

Why extract the program if it is not run? It documents the recipe. The solver then reproduces that logic in Python.

SOURCES

TAPO · SOLVE.PY · POWERPOINT SLIDES 17–18

LIMITATIONS AND PROVENANCE

Static inspection of the supplied script. No decryption, download or binary execution was performed for this presentation.

SLIDE 45 / 46 · 45–60 seconds

Tapo — the result

WHAT TO SAY

“The supplied flag.txt contains this exact text, identical to the one in the PowerPoint. We therefore have the result and a script describing how to recover it from the dump. The submission was confirmed by the platform: the challenge is solved. The camera kept its secret and its instructions in the same place.”

ANIMATION CUES

1. Introduce the challenge.
2. → Reveal the next idea.
3. → Close with the evidence or the lesson.

BACKGROUND

The received files are Resolution_CTF_Tapo.pptx, solve.py, ctf_unlock, flag.enc and flag.txt. flag.enc and flag.txt are each 35 bytes long. The flag matches the source presentation. The script checks the BA26{…} format, but that check is not server validation.

LIKELY QUESTIONS

Did we reproduce the solve here? No: the files were read and the script inspected. The dump was not supplied with this request and nothing was executed. The reported reproduction is the one in the source PowerPoint.

SOURCES

TAPO · SUPPLIED FLAG.TXT · SUBMISSION CONFIRMED BY THE USER

LIMITATIONS AND PROVENANCE

The match between flag.txt and the PowerPoint has been verified. The user confirmed submission to the platform after the PowerPoint was received; this confirmation replaces the source presentation's provisional status. The origin of the solution and its author are not specified; no agent attribution is invented.

Conclusion

WHAT TO SAY

“The attacks differ, but we can explain them through the same questions. What was the gate? Which assumption was weak? And what evidence shows that it worked? Moving from the result to the reasoning is how we make the agents’ work our own.”

ANIMATION CUES

1. Introduce the challenge.
2. → Reveal the next idea.
3. → Close with the evidence or the lesson.

BACKGROUND

The diagrams simplify the mechanisms. The notes connect each claim to the available journals and artifacts. Annotated screenshots produced after the event are identified as such.

LIKELY QUESTIONS

Chapters can be presented separately. For a short slot, start with embeddings, id_please and wrong_path. More specialized mechanisms are explained in the speaker notes.

SOURCES

SAVED JOURNALS AND ARTIFACTS · DOCUMENTED WALKTHROUGH

LIMITATIONS AND PROVENANCE

Teaching diagrams based on the files. Animation timings do not represent the actual time taken to solve the challenges.